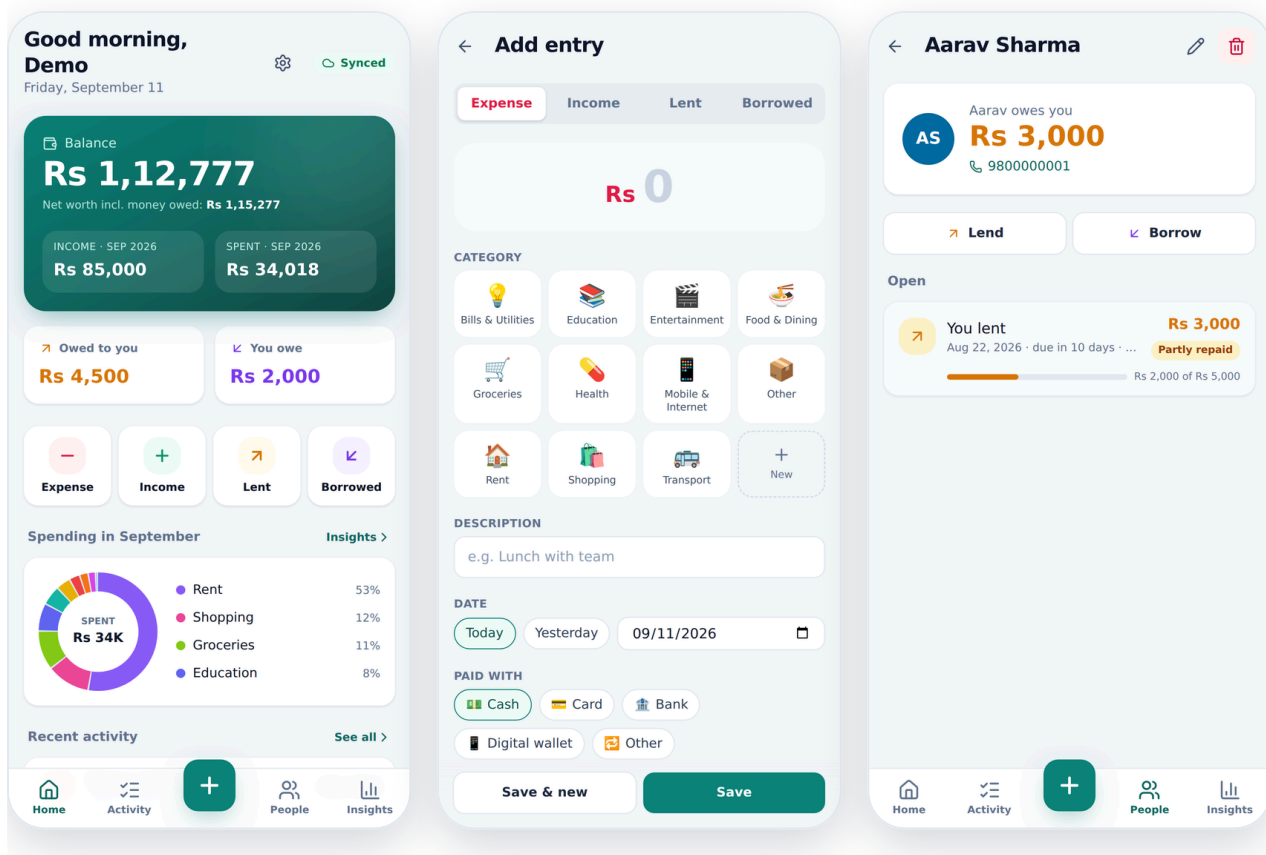




Project documentation

Version 1.0

An offline-first personal finance Progressive Web App — installable on Android, usable in any desktop browser — for tracking income, expenses, money lent and borrowed, and repayments.



Built

11 September 2026

Stack

React · Vite · Tailwind · IndexedDB · PWA · Node.js · Express · PostgreSQL · Prisma

Status

V1 complete · 62 automated tests + 20-step end-to-end run passing

Contents

01	Overview & screens	What V1 does
02	User guide	USER_GUIDE.md
03	Architecture	ARCHITECTURE.md
04	Offline-first & sync	SYNC.md
05	API reference	API.md
06	Security	SECURITY.md
07	Testing	TESTING.md
08	Running, deploying & installing	DEPLOYMENT.md
09	Roadmap	ROADMAP.md

01 Overview & screens

SpendWise answers five questions: *Where did my money go? How much do I have? Who owes me? Whom do I owe? Am I saving enough?* Every change is written to the device first (IndexedDB), so the app is fully usable without internet, and a background sync engine keeps each device and the server in step.

5

core record types:
transactions, categories,
people, loans, repayments

34

REST endpoints + 2 sync
endpoints

62

unit, API and offline-
engine tests

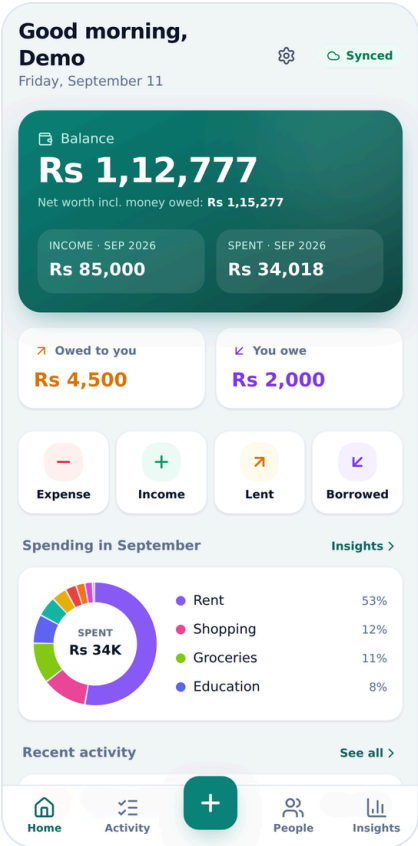
20

end-to-end steps incl.
offline → online

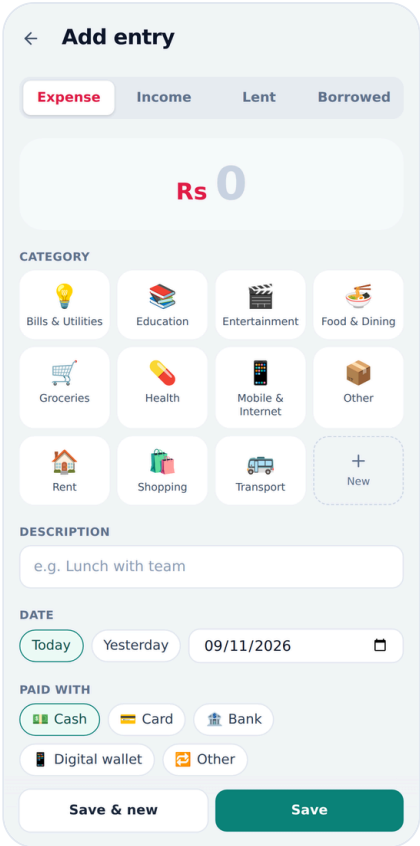
V1 features

Area	What you get
Accounts	Register / login, 15-min JWT + rotating 30-day refresh tokens, per-user data isolation, rate-limited login
Income & expenses	Amount, category, date, description, payment method (cash, card, bank, digital wallet), notes; edit, delete, search, month filter
Categories	16 defaults with icons and colours, plus custom categories
People & loans	Money lent / borrowed per person, due dates, overdue flags, net balance per person
Repayments	Partial or full; outstanding amount and status (open, partly repaid, overdue, settled)
Dashboard	Balance, net worth, month income & spending, owed to you / you owe, spending donut, recent activity, loans due soon
Insights	Spending by category, 6-month income vs expenses, daily spending, savings rate, change vs last month
PWA	Installable (Android, desktop, iOS), home-screen shortcuts, offline app shell, “new version” prompt
Offline & sync	Local-first writes, sync queue with pending / syncing / synced / failed states, retries with backoff, duplicate protection, deterministic multi-device conflict rules
UI	Mobile-first (bottom nav + big “+”), sidebar on desktop, light & dark themes, 12 currencies with NPR default and lakh grouping

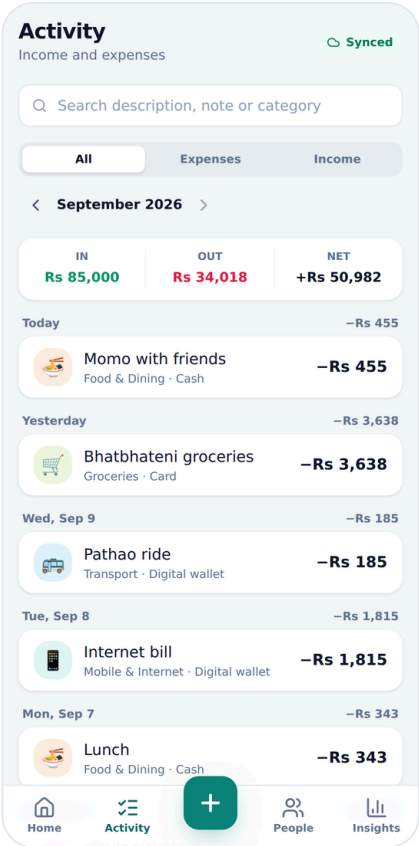
Phone screens



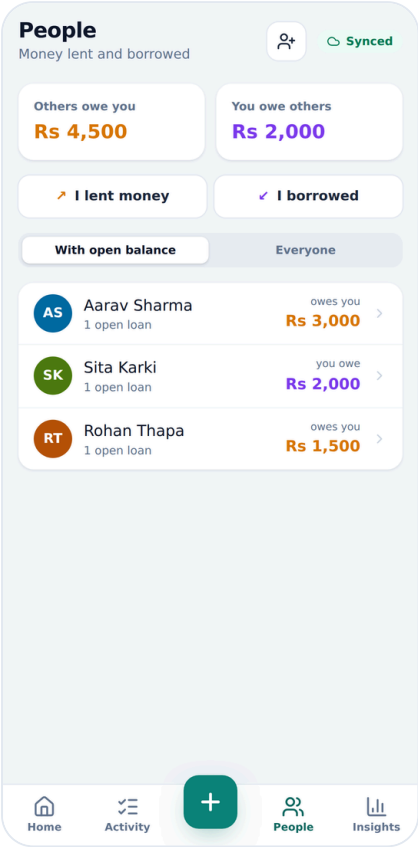
Dashboard



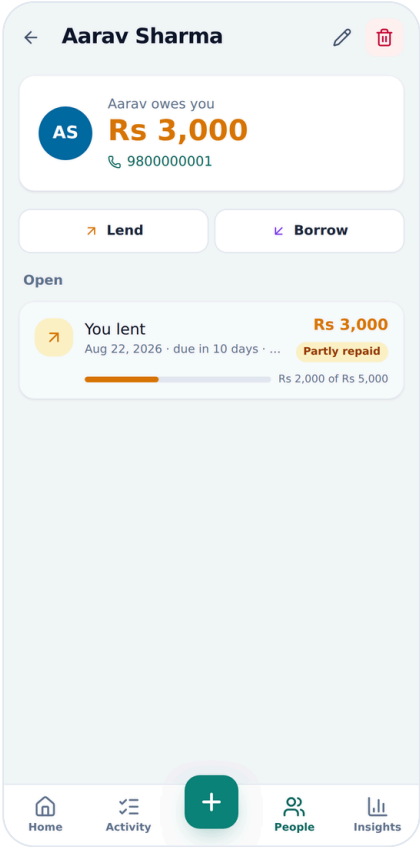
Add entry — fast, thumb-friendly



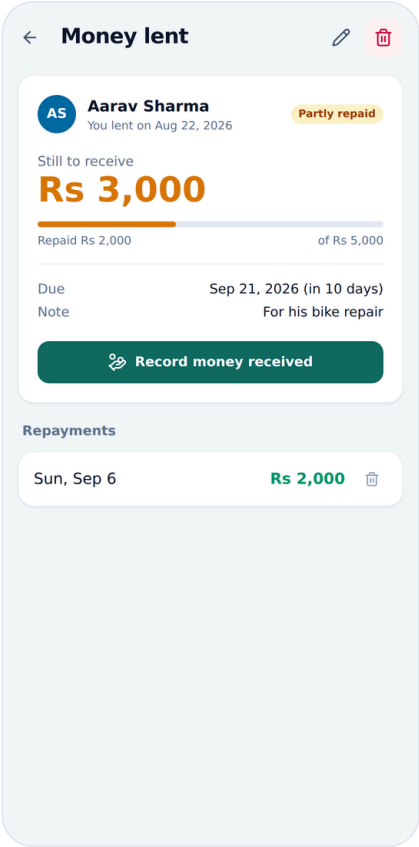
Activity — grouped by day



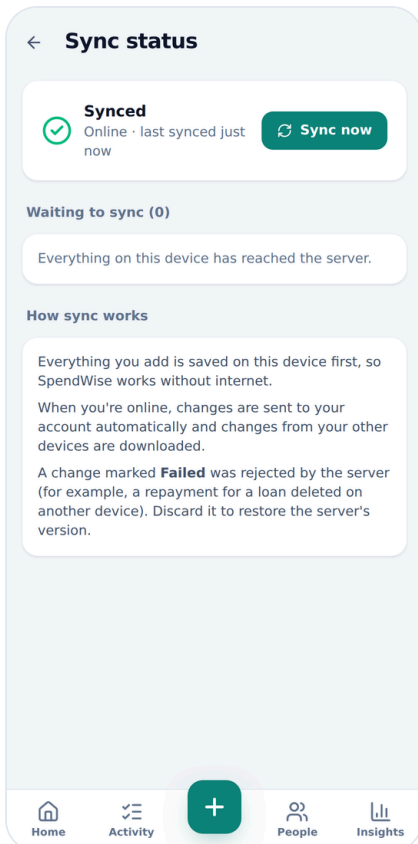
People — who owes whom



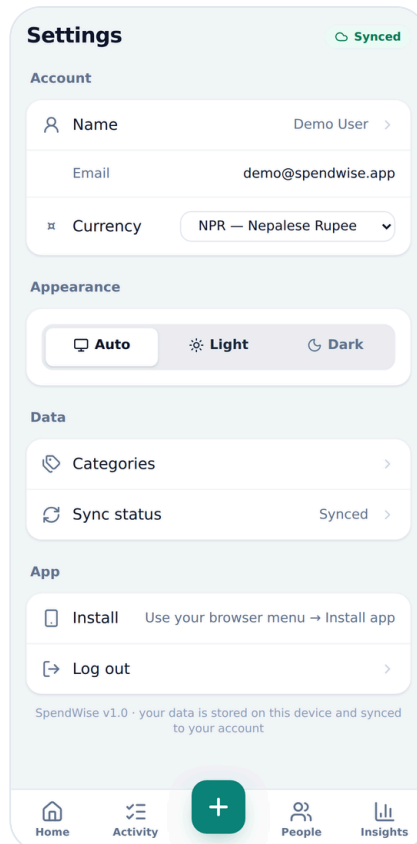
Person — open and settled loans



Loan — repayments and progress



Sync status

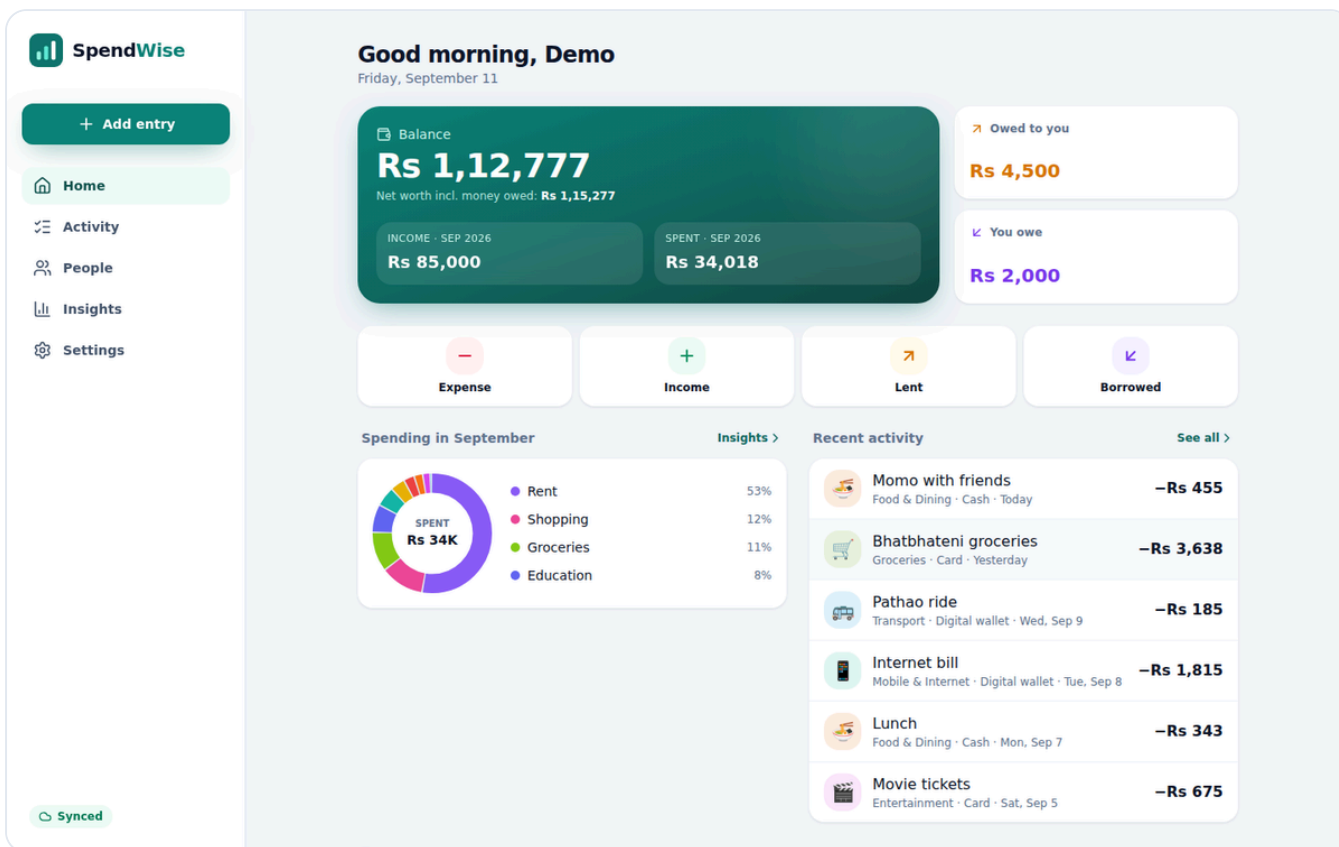


Settings

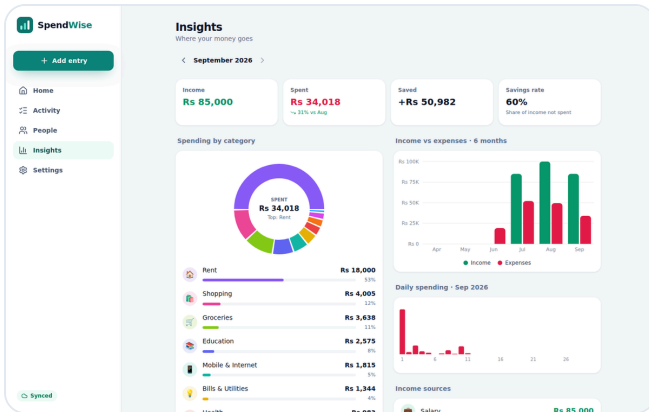


Dark theme

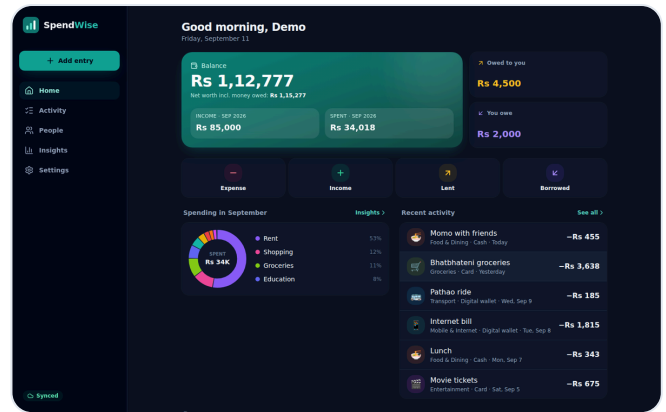
Desktop



Dashboard in a desktop browser (sidebar layout)



Insights



Dark theme

02 User guide

SpendWise answers five questions: **Where did my money go? How much do I have? Who owes me? Whom do I owe? Am I saving enough?**

Getting started

1. Open SpendWise and **Create an account** (name, email, password, currency).
2. On Android, tap **Install** when Chrome offers it (or  → *Install app*).
3. You'll land on **Home**. Everything you add from now on is saved on your phone first, so it works with no internet.

Adding money in and out — the + button

The big + in the middle of the bottom bar opens *Add entry*. Choose:

Tab	Use it for
Expense	anything you spent: amount → category → (description) → Save
Income	salary, business, freelance, gifts...
Lent	you gave someone money they'll return
Borrowed	someone gave you money you'll return

Tips for speed:

- The amount field is focused automatically with a number keypad.
- **Today / Yesterday** chips set the date in one tap.
- **Save & add another** keeps the screen open for several entries.
- Long-press the app icon → *Add expense* to jump straight in.

Home

- **Balance** — cash you have: income – expenses, adjusted for money lent/borrowed and repaid.
- **Net worth** — balance plus money owed to you minus money you owe.
- **Income / Spent** this month, **Owed to you**, **You owe**.
- Spending donut for the month, recent activity, and loans **due soon**.

Activity

All income and expenses grouped by day, with day totals. Switch months with < >, filter Expenses/Income, or search by description, note or category. Tap an entry to edit or delete it.

People — lending & borrowing

- The list shows everyone with an open balance ("owes you" / "you owe").
- Tap a person to see their loans, add a new one (**Lend / Borrow**), edit their details or delete them.
- Tap a loan → **Record money received** (or *Record repayment*) to log a partial or full repayment. **Full** fills in the whole outstanding amount.
- Loan status: *Open* · *Partly repaid* · *Overdue* (past its due date) · *Settled*.

Insights

Per month: income, spending (with change vs last month), amount saved and **savings rate**, spending by category, 6-month income vs expenses, and daily spending.

Settings

- Name and **currency** (NPR, INR, USD, EUR, GBP and more).
- **Theme**: Auto / Light / Dark.
- **Categories**: add your own (icon + colour), rename or delete.
- **Sync status**: what is waiting to sync, retry, or discard a failed change.
- **Install** and **Log out**.

Offline & sync — what the icons mean

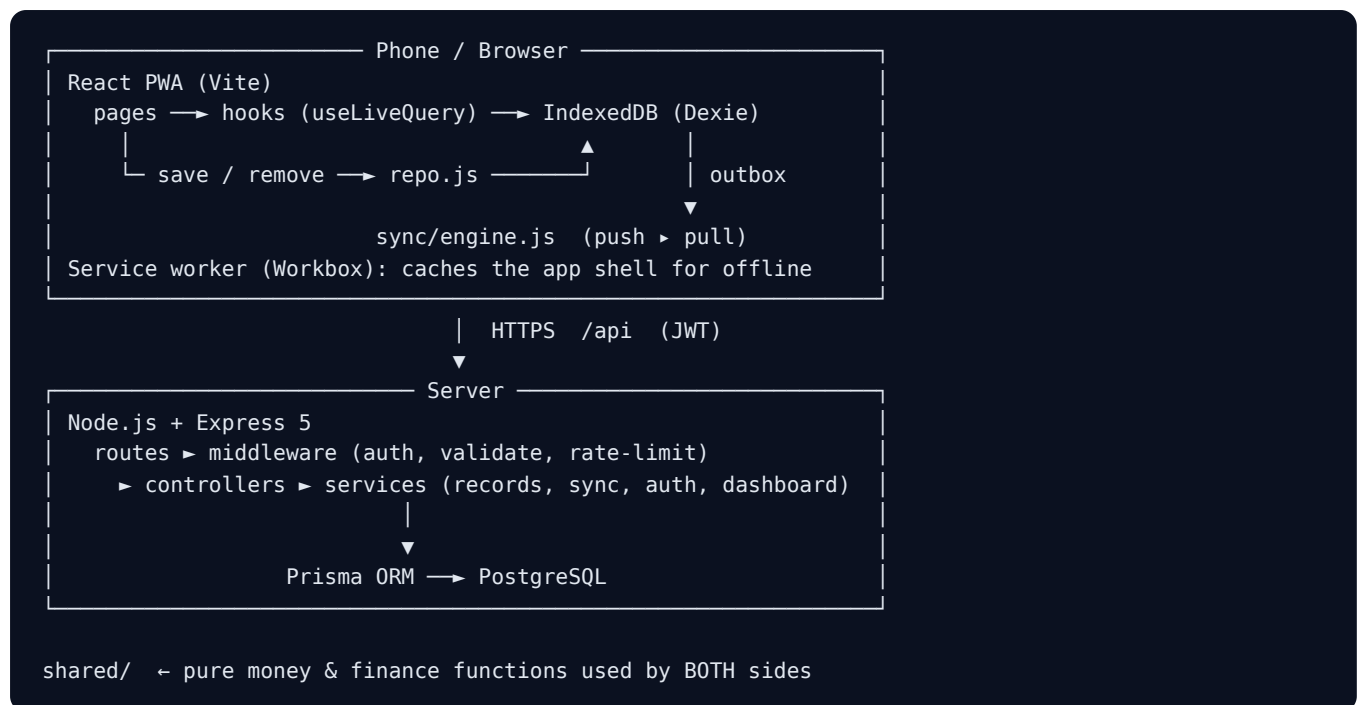
You see	Meaning
Synced badge	Everything is on the server
N pending	N changes saved on the phone, about to be sent
Offline · N to sync	No internet; changes are safe on the phone
▲↑ next to an item	That item hasn't reached the server yet
N failed	The server refused a change (e.g. the loan was deleted on another device). Open <i>Sync status</i> to retry or discard it
Sign in to sync	Your login expired; sign in again — nothing is lost

Using SpendWise on two devices (phone + laptop) with the same account keeps both in step automatically. If the same entry is edited on both while offline, the most recent edit wins; a deleted item stays deleted.

Privacy

Your records are visible only to your account. Logging out removes your data from that device (it stays safe on the server). If you have unsynced changes, SpendWise warns you first.

03 Architecture



The key difference from a normal CRUD app: **the client has its own database**. Screens never wait for the network. They read IndexedDB through live queries, and writes go to IndexedDB first. The sync engine reconciles with the server in the background. See [SYNC.md](#).

Repository layout

```
spendwise/
├── shared/           Pure JS used by frontend AND backend
│   └── src/          money.js · dates.js · finance.js · constants.js
├── backend/
│   ├── prisma/       schema.prisma · migrations/ · seed.js (demo account)
│   └── src/
│       ├── server.js  process entry: listen, housekeeping, shutdown
│       ├── app.js     Express app: headers, CORS, limits, routes, PWA
│       ├── config.js  environment configuration
│       ├── routes/    URL → controller mapping
│       ├── controllers/ HTTP layer (thin)
│       ├── services/  records (CRUD+ownership+conflicts), sync, auth...
│       ├── middleware/ requireAuth, validate (Zod), rate limit, errors
│       ├── validators/ Zod schemas for every request
│       └── utils/     errors, serialization, id checks
│   └── test/         Vitest + Supertest API tests (real PostgreSQL)
├── frontend/
│   ├── public/       icons, favicon, theme bootstrap, install screenshots
│   └── src/
│       ├── App.jsx    routes + auth guard
│       ├── pages/     Dashboard, AddEntry, Transactions, People, ...
│       ├── layouts/   AppLayout (bottom nav / desktop sidebar)
│       ├── components/ UI kit, forms, rows, charts, sheets, sync badge
│       ├── hooks/     useData, useFinance (live queries), session...
│       ├── db/        db.js (Dexie schema) · repo.js (writes+outbox)
│       ├── sync/      engine.js (push/pull, retries, conflicts)
│       ├── services/  api.js (Axios + token refresh), session, auth
│       └── utils/     formatting, navigation helpers
│   └── test/         Vitest + fake-indexeddb tests of the offline engine
├── e2e/              run.mjs (end-to-end) · screenshots.mjs
├── docs/             this documentation + PDF
├── Dockerfile, docker-compose.yml
└── package.json      npm workspaces: shared, backend, frontend
```

Technology choices

Layer	Choice	Why
UI	React 19 + Vite	Fast dev server, small production bundle
Styling	Tailwind CSS 4	Mobile-first utility classes, dark mode via <code>.dark</code> class
Routing	React Router 7	Client-side navigation, deep links work offline
HTTP	Axios	Interceptors for the token-refresh flow
Charts	Recharts	Donut + bar charts; lazy-loaded with the Insights page
Local DB	IndexedDB via Dexie 4	Transactions across tables, indexes, <code>useLiveQuery</code> for reactive screens
PWA	vite-plugin-pwa (Workbox)	Precached app shell, update prompt, web manifest
API	Node.js + Express 5	Async error handling built in
Validation	Zod 4	One schema per request, used by REST and sync

Layer	Choice	Why
Auth	JWT (15 min) + rotating refresh tokens (30 days), bcrypt	Works for an app that is offline for days
DB	PostgreSQL + Prisma 6	Relational integrity, migrations, typed queries

Data model

```

User ─┬─ Category      (income/expense, icon, colour; 16 defaults per user)
      │├─ Transaction  (income|expense, amount, category?, date, description, method, note)
      │├─ Person
      │ │├─ Loan        (lent|borrowed, amount, date, due date?, note)
      │ │└─ Repayment  (amount, date, note)
      │├─ RefreshToken  (hashed, expiry, revoked)
      └─ SyncOperation  (processed opIds – idempotency log)

```

Every financial table has `id` (client UUID), `clientUpdatedAt`, `updatedAt` (server), `deletedAt` (tombstone) and an index on `(userId, updatedAt)` for fast pulls. Loan status/outstanding are **computed**, never stored, so they can never disagree with the repayments.

Money

Amounts are integers in minor units (paisa/cents) end-to-end: input `"1,250.50"` → `125050` (`toMinor`), stored as `BIGINT`, displayed with `Intl.NumberFormat` (lakh grouping for NPR/INR: Rs 1,00,000). No floating-point arithmetic touches money.

Balance definitions

- **Balance** (cash you have) = income – expenses – money lent + money borrowed + repayments received – repayments paid
- **Net worth** = balance + owed to you – you owe (= income – expenses)
- **Owed to you / You owe** = sum of outstanding amounts on open loans

Implemented once in `shared/src/finance.js` and used by the dashboard (offline, on IndexedDB data) and by `/api/dashboard/*` (server).

04 Offline-first & sync

SpendWise treats the phone as the primary copy of your data and the server as the backup and the bridge between devices.

1. Local-first write

```
User taps Save
→ one IndexedDB transaction:
  • write the record (with _status = "pending")
  • append an op to the outbox
→ UI updates instantly (Dexie live queries)
→ sync engine is asked to run (debounced 400 ms)
  online → push now
  offline → wait
```

Code: `frontend/src/db/repo.js` (`saveRecord` , `deleteRecord`).

- Records get a **client-generated UUID**, so an expense created offline keeps the same id forever — no id remapping after sync.
- Repeated edits of a record that has **not been sent yet** are merged into the one waiting op. An op that was already attempted is never merged (the server may have applied it even though the phone never saw the reply).
- Deletes cascade locally exactly like on the server (person → loans → repayments).

2. The outbox (sync queue)

Table `outbox` in IndexedDB, one row per change:

field	meaning
<code>seq</code>	auto-increment — defines the order ops are sent in
<code>opId</code>	UUID — the idempotency key
<code>entity</code> , <code>action</code> , <code>recordId</code> , <code>data</code>	what to do
<code>clientUpdatedAt</code>	when the user made the change (for conflict resolution)
<code>status</code>	<code>pending</code> → <code>syncing</code> → removed when synced · or <code>failed</code>
<code>attempts</code> , <code>nextAttemptAt</code> , <code>permanent</code> , <code>error</code>	retry bookkeeping

Sync states shown to the user

State	Where you see it
pending — not yet on the server	cloud-upload icon on the item, "N pending" badge
syncing — being sent	spinning badge

State	Where you see it
synced — server accepted it	"Synced" badge, no icon on the item
failed — will be retried, or was rejected	"N failed" badge, details in <i>Settings</i> → <i>Sync status</i>

3. Push

`frontend/src/sync/engine.js` → `push()`

1. Take outbox ops in `seq` order (max 100 per request). Skip ops that were permanently rejected; stop at an op that is still backing off (keeps order, so a repayment is never sent before its loan).
2. Mark them `syncing`, `POST /api/sync/push`.
3. For each result:
 - `applied / stale` → delete the op; if no newer local change is queued for that record, store the server's copy and mark it `synced`.
 - `rejected` → keep the op as **failed (permanent)** with the server's message; the record is marked failed. The user can *Discard* it.
 - `error` (server hiccup) → failed, retried with backoff; after 8 server errors it becomes permanent.
4. If the whole request fails (no network, 5xx, timeout) every op in the batch becomes `failed` with **exponential backoff**: 2 s, 4 s, 8 s ... max 5 min. Network failures never count toward the permanent limit.

4. Pull

`GET /api/sync/pull?since=<cursor>` returns everything changed on the server since the last pull — including **tombstones** (`deletedAt`) so deletions reach every device. The cursor is the *server's* clock (never the phone's).

Merging rules (`applyPulled`):

- A record that still has a queued local change is **not overwritten** — the local edit wins until it is pushed; the server then decides.
- Tombstones delete the local copy.
- A *full* pull (first login on a device, or after discarding a failed change) also removes local records the server doesn't have (unless queued).

The server re-sends a 30-second overlap window on every pull, so a write that was still committing during the previous pull is never missed.

5. When does sync run?

- right after every local change (debounced),
- on app start,
- when the browser fires `online`,
- when the app comes back to the foreground,
- every 60 s while visible,
- when the user taps **Sync now**.

Only one sync runs at a time; calls during a run schedule exactly one more.

6. Duplicate-sync protection

The server stores every processed `opId` (table `SyncOperation`, primary key `(userId, opId)`) together with its result. If an op arrives again — because the response was lost, the app was killed mid-sync (ops stuck in `syncing` are reset to `pending` on the next start), or two tabs raced — the server returns the original result with `duplicate: true` and applies nothing. A repayment re-sent three times is counted once. Tested in `backend/test/sync.test.js` and `frontend/test/sync.test.js`.

7. Conflict rules (multi-device)

Deterministic, never silent:

1. **Ownership first.** A record id belongs to one user. Ops touching another user's record are rejected as "not found" (no information leak).
2. **Deletes are final.** Once deleted (tombstoned), later edits for that record are ignored (`stale`, reason `deleted`) — nothing is resurrected.
3. **Last write wins by `clientUpdatedAt`.** If the server already holds a version edited later than the incoming one, the incoming edit is ignored (`stale`, reason `newer-version-exists`) and the server copy is returned so the device converges.
4. **Business rules still apply.** A repayment for a loan deleted on another device, or one that would exceed the outstanding amount, is `rejected`.

Client clocks set far in the future are clamped to server time + 5 minutes so a wrong clock can't make a device's edits win forever.

Known V1 limits: whole-record last-write-wins (no field-level merge); clock skew within 5 minutes can still decide close races; pull returns all changes in one response (fine for personal-scale data).

8. Sessions while offline

The app opens without a network using the saved session. If the refresh token is rejected when the device reconnects (expired after 30 days, or revoked), the session is marked *expired*: **local data and queued changes are kept**, a banner asks the user to sign in again, and syncing resumes after sign-in. Logging out deletes that account's local database (after warning if there are unsynced changes).

05 API reference

Base URL: `/api` (same origin as the app, or `VITE_API_URL` when hosted separately).

- All request and response bodies are JSON.
- **Amounts are integers in minor units** (paise / cents): Rs 1,250.50 → `125050` .
- **Dates** are calendar dates `YYYY-MM-DD` ; timestamps are ISO-8601 strings.
- Every endpoint except `/health` and `/auth/register|login|refresh|logout` needs `Authorization: Bearer <accessToken>` .
- Record ids are UUIDs. Clients may supply their own id when creating a record (this is how offline-created records keep their identity).

Errors

```
{ "error": { "code": "VALIDATION_ERROR", "message": "amount: must be greater than zero", "details": [{ "path": "amount", "message": "must be greater than zero" }] } }
```

Status	Code	Meaning
400	<code>VALIDATION_ERROR</code>	Body/query failed validation or a business rule (e.g. over-repayment)
401	<code>UNAUTHORIZED</code> / <code>INVALID_TOKEN</code>	Missing or invalid access token
401	<code>TOKEN_EXPIRED</code>	Access token expired → call <code>/auth/refresh</code>
401	<code>SESSION_EXPIRED</code>	Refresh token invalid/used/expired → log in again
401	<code>INVALID_CREDENTIALS</code>	Wrong email or password
404	<code>NOT_FOUND</code>	Record does not exist or belongs to another user
409	<code>EMAIL_TAKEN</code> / <code>CONFLICT</code> / <code>STALE</code>	Duplicate email, duplicate id, or a newer version exists
422	<code>INVALID_REFERENCE</code>	Referenced category/person/loan is missing, deleted, not yours, or wrong type
429	<code>RATE_LIMITED</code>	Too many requests

Auth

Method	Path	Body	Returns
POST	<code>/auth/register</code>	<code>{ name, email, password (≥8), currency? }</code>	<code>201 { user, accessToken, refreshToken }</code> — also creates 16 default categories
POST	<code>/auth/login</code>	<code>{ email, password }</code>	<code>{ user, accessToken, refreshToken }</code>
POST	<code>/auth/refresh</code>	<code>{ refreshToken }</code>	New <code>{ user, accessToken, refreshToken }</code> ; the old refresh token stops working (rotation)

Method	Path	Body	Returns
POST	/auth/logout	{ refreshToken }	204 — refresh token revoked
GET	/auth/me	—	{ user }
PATCH	/auth/me	{ name?, currency? }	{ user }

Access tokens live 15 minutes, refresh tokens 30 days (configurable). Login and register are rate-limited to 20 requests per 15 minutes per IP.

Categories · People · Transactions · Loans

Each resource supports the same five operations:

Method	Path	Notes
GET	/ {resource}	List (deleted records excluded)
POST	/ {resource}	Create; optional client id
GET	/ {resource} /:id	One record
PATCH	/ {resource} /:id	Partial update (merged, then fully re-validated)
DELETE	/ {resource} /:id	Soft delete → 204 (idempotent)

Resources: categories, people, transactions, loans.

Record shapes

```
// Category
{ "id": "uuid", "name": "Food & Dining", "type": "expense", "icon": "🍽️", "color": "#f97316",
  "isDefault": true }

// Transaction
{ "id": "uuid", "type": "expense", "amount": 25050, "categoryId": "uuid|null", "date": "2026-09-11",
  "description": "Lunch", "paymentMethod": "cash|card|bank|wallet|other|null", "note": null }

// Person
{ "id": "uuid", "name": "Aarav Sharma", "phone": "98...", "note": null }

// Loan (GET responses add repaid, outstanding, status, progress)
{ "id": "uuid", "personId": "uuid", "direction": "lent|borrowed", "amount": 500000,
  "date": "2026-09-01", "dueDate": "2026-10-01|null", "note": null,
  "repaid": 200000, "outstanding": 300000, "status": "open|partial|overdue|settled", "progress":
  0.4 }

// Repayment
{ "id": "uuid", "loanId": "uuid", "amount": 200000, "date": "2026-09-05", "note": null }
```

Every record also carries createdAt, updatedAt, clientUpdatedAt, deletedAt.

Transaction filters

```
GET /transactions?type=expense&from=2026-09-01&to=2026-09-30&categoryId=...  
&q=coffee&limit=200&offset=0
```

Business rules enforced by the server

- A transaction's category must be the same type (an expense can't use "Salary").
- Loan due date can't be before the loan date.
- A repayment can't exceed the loan's outstanding amount.
- A loan's amount can't be reduced below what has already been repaid.
- Deleting a person deletes their loans and repayments; deleting a loan deletes its repayments.
- Deleted categories stay referenced by old transactions (shown as uncategorized).

Repayments

Method	Path
GET	/loans/:id/repayments
POST	/loans/:id/repayments — { amount, date, note? }
GET / PATCH / DELETE	/repayments/:id

Dashboard

Method	Path	Returns
GET	/dashboard/summary? month=YYYY-MM	balance, netWorth, totalIncome, totalExpense, periodIncome, periodExpense, periodNet, savingsRate, owedToMe, iOwe, recent[]
GET	/dashboard/analytics? month=YYYY-MM&months=6	monthly[] (income/expense per month), expenseByCategory[], incomeByCategory[]

`balance` = income – expenses – money lent + money borrowed + repayments received – repayments paid (cash you actually have). `netWorth` = balance + owedToMe – iOwe.

Sync (used by the offline app)

POST /sync/push

```
{ "ops": [  
  { "opId": "uuid", "entity": "transactions", "action": "upsert", "recordId": "uuid",  
    "data": { "type": "expense", "amount": 25000, "date": "2026-09-11" },  
    "clientUpdatedAt": "2026-09-11T08:15:00.000Z" },  
  { "opId": "uuid", "entity": "people", "action": "delete", "recordId": "uuid",  
    "clientUpdatedAt": "2026-09-11T08:16:00.000Z" }  
]
```

Up to 200 ops, applied in order. Response:

```
{ "serverTime": "...", "results": [
  { "opId": "...", "status": "applied", "record": { ... } },
  { "opId": "...", "status": "stale", "reason": "newer-version-exists", "record": { ...server copy... } },
  { "opId": "...", "status": "rejected", "error": "Repayment exceeds the outstanding amount of 3000" },
  { "opId": "...", "status": "applied", "duplicate": true }
] }
```

GET /sync/pull?since=<serverTime>

Without `since`: full snapshot of non-deleted records. With `since`: every record changed after it (a 30-second overlap is included), **including deletions** (`deletedAt` set). Response: `{ serverTime, full, changes: { categories, people, transactions, loans, repayments } }`.

See [SYNC.md](#) for the full protocol and conflict rules.

Health

GET /health → `{ "status": "ok", "time": "..." }`

06 Security

How each requirement from the project plan is implemented.

Requirement	Implementation
Hash passwords; never store plain text	<code>bcrypt</code> (cost 12) via <code>bcryptjs</code> ; only <code>passwordHash</code> is stored. Login compares against a dummy hash for unknown emails so response time doesn't reveal which emails exist.
Authenticate API requests	<code>requireAuth</code> middleware verifies a signed HS256 JWT (15-minute lifetime) on every non-auth route.
Authorize access to user-owned records	Every query is scoped by <code>userId</code> . Records of another user are reported as 404 Not found (no information leak). References (category, person, loan) must belong to the same user. Tested in <code>records.test.js</code> and <code>sync.test.js</code> .
Validate amounts, dates, IDs and bodies on the server	Zod schemas for every request (<code>backend/src/validators/schemas.js</code>): integer positive amounts with a cap, real calendar dates, UUIDs, length limits, enums. Malformed ids never reach the database.
Never trust client-side validation alone	Sync ops go through the same schemas and business rules as the REST API (over-repayment, category type, due date $\geq$ loan date...).
Secrets in environment variables	<code>DATABASE_URL</code> , <code>JWT_SECRET</code> come from env; the server refuses to start in production with a missing/short/default <code>JWT_SECRET</code> . <code>.env</code> is git-ignored.
HTTPS in production	Required for PWA install anyway; helmet sends HSTS. Deployment guide covers hosts with automatic HTTPS.
Prevent access to other users' private records	See authorization row; the sync pull only ever returns the caller's rows.
Rate-limit sensitive auth endpoints	<code>/auth/login</code> and <code>/auth/register</code> : 20 requests / 15 min per IP; whole API: 600 / min. Set <code>TRUST_PROXY</code> behind a load balancer.
Don't send unnecessary data to third-party AI	V1 has no AI integration — no financial data leaves your server.
Plan data deletion/export	Deleting an account cascades all its rows (<code>onDelete: Cascade</code>); a user-facing export/delete flow is planned for V2.

Tokens and sessions

- **Access token:** JWT, 15 min, sent as `Authorization: Bearer`.
- **Refresh token:** 48 random bytes, 30 days, stored **only as a SHA-256 hash, single-use** (rotated on every refresh with an atomic conditional update, so two concurrent refreshes can't both succeed), revoked on logout, pruned daily.
- The PWA keeps tokens in `localStorage` so it can open offline. Because of that, the page ships a strict Content-Security-Policy (`script-src 'self'`, no inline scripts, `connect-src 'self'`, `object-src 'none'`, `frame-ancestors 'self'`) to make XSS token theft much harder.

Other hardening

- `helmet` security headers; `x-powered-by` disabled.
- API responses are `Cache-Control: no-store`; the service worker never caches `/api`.
- JSON body limit 1 MB; sync batches limited to 200 ops.
- Soft deletes are final — a deleted record can't be resurrected through sync.
- Client clocks are clamped (max +5 min) so a device with a wrong clock can't win every conflict.
- Logging out deletes the account's IndexedDB database from the device.
- CORS is off by default (same-origin); enable only for your own frontend domain.

Known limitations (V1)

- Tokens in `localStorage` (mitigated by CSP) rather than `httpOnly` cookies — a deliberate trade-off for offline-first startup.
- No email verification / password reset yet (needs an email provider — V2/V5).
- Rate limiting uses in-memory counters; use a shared store (e.g. Redis) when running more than one server instance.

07 Testing

SpendWise has four layers of automated tests. All of them pass on the V1 codebase (**62 unit/API tests + a 20-step end-to-end run**).

Layer	Tool	Where	What it proves
Financial logic	Vitest	<code>shared/test/finance.test.js</code> (14)	Money parsing without float errors, lakh formatting, balances, loan status/outstanding, per-person balances, analytics
API	Vitest + Supertest + real PostgreSQL	<code>backend/test/*.test.js</code> (35)	Auth, validation, CRUD, business rules, user isolation , sync idempotency & conflicts
Offline engine	Vitest + fake-indexeddb	<code>frontend/test/sync.test.js</code> (13)	Local-first writes, outbox, retries/backoff, duplicate protection, merge rules
End-to-end	Playwright (Chromium, Pixel 7 + desktop)	<code>e2e/run.mjs</code>	The real app: service worker, offline mode, reconnect sync, multi-device

Run: `npm test` (first three) and `npm run test:e2e` (needs the app running — see [DEPLOYMENT.md](#)).

Coverage of the project's testing strategy

Requirement (from the project plan)	Covered by
Unit-test financial calculations and validation	<code>finance.test.js</code> ; <code>records.test.js</code> > <code>rejects invalid input</code> (7 cases)
API-test authentication and authorization	<code>auth.test.js</code> — register, login, duplicate email, bcrypt hash stored, expired/forged tokens, refresh rotation, logout revocation
CRUD for each core entity	<code>records.test.js</code> — transactions, people, loans, repayments, categories
Offline creation, editing and deletion	<code>frontend/test/sync.test.js</code> > <code>local-first writes</code> ; E2E steps 8–10
Reconnect and sync behaviour	engine test "stays offline... then syncs on reconnect"; E2E step 11–12
Retry after failed synchronization	engine tests "keeps ops queued with backoff", "marks rejected ops as permanently failed...", "discarding a failed change..."
Duplicate-sync protection	API "is idempotent: re-sending the same ops never applies them twice" (incl. a repayment sent twice counted once); engine "does not duplicate records when a response is lost", "re-queues ops interrupted mid-sync"

Requirement (from the project plan)	Covered by
Mobile layouts on Android screen sizes	E2E runs on a Pixel 7 profile; screenshots in docs/screenshots/
PWA installation & service-worker behaviour	E2E step 4 (SW controls page), 5 (manifest installable, maskable icon), 8 (offline reload served from SW cache)
User isolation	API "never lets one account read or change another account's records", sync "never lets another account overwrite or delete...", pull "does not leak another account's changes"

End-to-end run (latest)

```

01. Unauthenticated visit redirects to /login
02. Registered a new account and landed on the dashboard
03. Initial sync completed (default categories downloaded)
04. Service worker installed and controlling the page
05. Web app manifest OK (SpendWise – Personal Finance, 3 icons, maskable icon present)
06. Added an expense online – it synced
07. Added income – synced
08. Went OFFLINE and reloaded – app shell served by the service worker, data read from IndexedDB
09. Added an expense while offline ("Saved on this device")
10. Lent money to a NEW person while offline – badge: "Offline · 3 to sync"
11. Back ONLINE – pending changes synced automatically
12. Verified on the server: 3 transactions, 1 person, 1 loan (Rs 5,000)
13. Recorded a partial repayment – outstanding Rs 3,000 on the server
14. Dashboard, Activity and Insights render
15. Signed in on a second "device" (desktop, dark mode) – all data pulled from the server
16. Edit made on desktop appeared on the phone after sync
17. Changed currency to USD – amounts re-render and the setting synced to the account
18. Created a custom category – synced to the server
19. Logged out – this device's local database was deleted
20. No JavaScript errors in the console
✅ All E2E checks passed.

```

Bugs the tests caught during development

- Mobile dashboard cards overflowed the screen width (CSS grid `min-width:auto`) — fixed with `grid-cols-1 / min-w-0`; seen in E2E screenshots.
- Compact amounts used Indian-English "T" for thousand ("Rs 1.3T") — now "Rs 1.3K".
- Amounts with paisa showed one decimal ("Rs 80,689.5") — now "Rs 80,689.50".
- `upgrade-insecure-requests` CSP broke plain-http use on a LAN IP — removed (HTTPS + HSTS in production instead).
- The login rate limiter correctly blocked sign-ins after many E2E runs from one IP.

Manual checklist before a release

- ☐ Install on a real Android phone from the HTTPS URL; open from the home screen.
- ☐ Airplane mode: open the app, add an expense and a loan, close and reopen, turn airplane mode off → badge goes to *Synced*.
- ☐ Two devices, same account: edit on one, see it on the other.

- ☐ Deploy a new build → "A new version is ready" prompt appears → Update works.
- ☐ Dark mode and a small phone (360 px wide).

08 Running, deploying & installing

1. Local development

Requirements: **Node.js 22.9+** (24 LTS works) and **PostgreSQL 14+** (16 recommended).

```
# 1. install everything (npm workspaces)
npm install

# 2. create a database and configure the API
createdb spendwise          # or use Docker: docker compose up -d db
cp backend/.env.example backend/.env  # edit DATABASE_URL and JWT_SECRET

# 3. create the tables
npm run db:migrate

# 4. (optional) load a demo account with 3 months of data
npm run db:seed              # → demo@spendwise.app / demo1234

# 5. start API (:4000) and web app (:5173) together
npm run dev
```

Open <http://localhost:5173>. Vite proxies `/api` to the API on port 4000.

The service worker is only active in production builds. To test offline mode and installation locally: `npm run build && npm start`, then open <http://localhost:4000>.

On Windows (PowerShell)

1. Install **Node.js 22+** (you have it if `node -v` works) and **PostgreSQL 16** from [postgresql.org](https://www.postgresql.org) (remember the password you set for the `postgres` user) — or install Docker Desktop and run `docker compose up -d db` instead.
2. Create the database (PostgreSQL installer route): open *SQL Shell (psql)* and run `CREATE DATABASE spendwise;`
3. In the project folder:

```
npm install
Copy-Item backend\.env.example backend\.env
notepad backend\.env          # set DATABASE_URL, e.g.
postgresql://postgres:YOURPASSWORD@localhost:5432/spendwise
npm run db:migrate
npm run db:seed                # optional demo data
npm run dev                    # http://localhost:5173
```

Running the tests locally

The API tests need a separate, throwaway database whose name contains `test` :

```
createdb spendwise_test # or CREATE DATABASE spendwise_test;
cp backend/env.test.example backend/.env.test # edit DATABASE_URL if needed
npm test # shared + backend + frontend
```

End-to-end browser test: `npm run build && npm start`, then in a second terminal `npx playwright install chromium` (first time only) and `npm run test:e2e`.

2. Production build (one server)

```
npm ci
npm run build # builds frontend/dist
npm run db:deploy # apply migrations
NODE_ENV=production npm start
```

The Express server serves the API under `/api` and the built PWA from `frontend/dist` on the same origin — no CORS needed, and cookies/tokens stay on one domain.

Required environment variables (backend)

Variable	Example	Notes
<code>DATABASE_URL</code>	<code>postgresql://user:pass@host:5432/spendwise</code>	Required
<code>JWT_SECRET</code>	48+ random bytes	Required in production (≥ 32 chars). <code>openssl rand -base64 48</code>
<code>PORT</code>	4000	
<code>NODE_ENV</code>	production	
<code>TRUST_PROXY</code>	1	Set when behind a load balancer (Render, Railway, Fly, Nginx) so rate limiting sees real IPs
<code>CORS_ORIGIN</code>	<code>https://spendwise.example.com</code>	Only if the frontend is on another domain
<code>SERVE_CLIENT</code>	true	false if you deploy the frontend separately
<code>ACCESS_TOKEN_TTL</code> / <code>REFRESH_TOKEN_DAYS</code>	15m / 30	Optional

3. Docker

```
JWT_SECRET=$(openssl rand -base64 48) docker compose up --build
```

Starts PostgreSQL and the app on <http://localhost:4000>. Migrations run automatically on container start.

4. Hosting options

Option A — one service (simplest). Any Node host with a PostgreSQL add-on: Render, Railway, Fly.io, a VPS.

- Build command: `npm ci && npm run build`
- Start command: `npm run db:deploy && npm start`
- Set `DATABASE_URL`, `JWT_SECRET`, `NODE_ENV=production`, `TRUST_PROXY=1`.
- Managed Postgres: Neon, Supabase, Render, Railway all work.

Option B — split. Frontend on Vercel/Netlify/Cloudflare Pages, API elsewhere.

- Frontend build: `npm ci && npm run build -w frontend`, output `frontend/dist`, env `VITE_API_URL=https://api.example.com/api`, and a SPA fallback rule (all routes → `/index.html`).
- API: `SERVE_CLIENT=false`, `CORS_ORIGIN=https://your-frontend.example.com`.

HTTPS is required for installation as an app and for service workers (except on `localhost`). All the hosts above provide it automatically.

5. Installing on Android (PWA)

1. Open your SpendWise URL in **Chrome** on the phone.
2. Chrome shows an **Install app** banner — or open the `⋮` menu → **Install app** (older versions: **Add to Home screen**). SpendWise → Settings → *Install SpendWise* also triggers it.
3. SpendWise appears in the app drawer with its own icon, opens full-screen without the browser bar, and works offline.
4. Long-press the icon for the **Add expense / Add income** shortcuts.

Testing on a phone before deploying: the phone needs HTTPS, so expose your local server with a tunnel, e.g. `npx cloudflared tunnel --url http://localhost:4000` or `ngrok http 4000`, and open the https URL on the phone.

Play Store (optional, later): wrap the PWA in a Trusted Web Activity with [Bubblewrap](#) — no code changes needed.

6. Installing on desktop & iPhone

- **Chrome / Edge (Windows, macOS, Linux):** install icon in the address bar, or menu → *Install SpendWise*.
- **Safari (iPhone/iPad):** Share → *Add to Home Screen*.
- Or simply use it in any modern browser — it's the same app.

7. Updating

Deploy a new build. Open apps download it in the background and show "A new version of SpendWise is ready → Update", so nobody loses a half-typed entry.

8. Backups

All server data lives in PostgreSQL — use your host's automated backups or `pg_dump`. Each device also holds a full local copy of its user's data.

09 Roadmap

V1 (this release) is the **personal finance MVP**. The codebase was shaped so the next versions slot in without rewrites.

✓ V1 — Personal Finance MVP (done)

Authentication · income & expense tracking · categories · people · lending & borrowing · repayments · dashboard · basic analytics · mobile-first UI · PWA installability · offline creation & local persistence · cloud synchronization.

Definition of done met: implementation exists, happy paths and invalid input are tested, offline behaviour is covered, user isolation is tested, the UI is integrated, and documentation is written.

V2 — Smart Finance (next)

Feature	Where it plugs in
Budgets per category/month	New <code>Budget</code> model + entity config in <code>records.service.js</code> ; progress bars on the dashboard from <code>categoryBreakdown</code>
Savings goals & contributions	<code>SavingsGoal</code> / <code>Contribution</code> models, same sync pattern as loans/repayments
Recurring transactions	A <code>Recurrence</code> rule on transactions; materialise due items locally on app open
Custom date ranges & advanced filters	Transactions page already has search/type/month — add range & category filters
CSV / JSON export & import	Client-side from IndexedDB (works offline)
Notifications & reminders (due loans)	Web Push via the existing service worker
Field-level conflict merge	Replace whole-record last-write-wins in <code>upsertRecord</code>

V3 — AI Finance Assistant

Natural-language entry ("lunch 250 yesterday"), category suggestions, spending summaries, "ask your data" questions, unusual-spending alerts. Keep the principle *AI-assisted, not AI-dependent*: send only the minimum aggregated data to the model, keep an offline rule-based parser as fallback.

V4 — Connected money between users

Friend invitations, loan requests that the other person accepts/declines, shared loan status & repayment history. Needs a `LoanShare` relationship with explicit permissions — the current ownership checks in `records.service.js` are the place to extend.

V5 — Product / SaaS

Onboarding, subscription tiers, multi-device management, support, product analytics, email verification & password reset, Redis-backed rate limiting, horizontal scaling, terms/privacy/security pages, account export & deletion.

Ideas backlog

Multiple wallets (cash / bank / eSewa-Khalti balances) · receipt OCR · currency conversion · family/shared budgets · PDF/CSV reports · bank statement import · forecasting · group expenses & settlements · Play Store listing via Trusted Web Activity (Bubblewrap).